

Python is one of the most popular programming languages. It's simple to use, packed with features and supported by a wide range of libraries and frameworks. Its clean syntax makes it beginner-friendly.

- A high-level language, used in data science, automation, AI, web development and more.
- Known for its readability, which means code is easier to write, understand and maintain.
- Backed by strong library support, we don't have to build everything from scratch.

Basic Code Example

The following is a simple program that displays the message "Hello, World!" on the screen.

```
print("Hello World!")
```

Output

Hello World!

Why Learn Python?

- Requires fewer lines of code compared to other programming languages like Java.
- Provides libraries and frameworks such as Django and Flask for web development and tools like Pandas, TensorFlow and Scikit-learn for artificial intelligence, machine learning and data analysis.
- Cross-platform, works on Windows, Mac and Linux without major changes.
- Used by top tech companies like Google, Netflix and NASA.
- Many Python coding job opportunities in Software Development, Data Science and AI/ML.

Basics

In this section, we'll cover the basics of Python programming, including installation, writing first program, understanding comments and working with variables, keywords and operators. Before starting to learn python we need to [install python](#) on our system.

- [Introduction](#)
- [Applications](#)
- [Input and Output](#)
- [Variables](#)
- [Operators](#)
- **Quiz:** [Basics](#), [I/O](#)
- [Keywords](#)
- [Data Types](#)
- **Quiz:** [Data Types](#), [Numbers](#), [Boolean](#)
- [Conditional Statements](#)
- [Loops](#)
- **Quiz:** [Control Flow](#), [Loops](#)

Functions

In this section of Python 3 tutorial we'll explore Python function syntax, parameter handling, return values and variable scope. Along the way, we'll also introduce versatile functions like range(), map, filter and lambda functions.

- [Functions](#)
- [Pass in Functions](#)
- [Global and Local Variables](#)

- [Recursion](#)
- [*args and **kwargs in Function](#)
- [First Class Function](#)
- [Lambda Function](#)
- [Map, Reduce and Filter Function](#)
- [Inner Function](#)
- [Decorators](#)
- **Quiz:** [Functions](#)

Data Structures

Python offers versatile collections of data types, including lists, string, tuples, sets, dictionaries and arrays. In this section, we will learn about each data types in detail.

- [Strings](#)
- [List](#)
- **Quiz:** [List](#), [String](#)
- [Tuples](#)
- [Dictionary](#)
- **Quiz:** [Tuples](#), [Dictionary](#)
- [Sets](#)
- [Arrays](#)
- [List Comprehension](#)
- **Quiz:** [Sets](#), [Arrays](#), [List Comprehension](#)

Python's collections module offers essential data structures, including the following:

- [Counters](#)
- [Heapq](#)
- [Deque](#)
- [OrderedDict](#)
- [Defaultdict](#)
- **Quiz:** [Counters](#), [Heapq](#), [Deque](#), [OrderedDict](#)
- Complete Tutorial on [DSA with Python](#)

OOP Concepts

In this section, we'll explore the core principles of object-oriented programming (OOP). From encapsulation to inheritance, polymorphism, abstract classes and iterators, we'll cover the essential concepts that helps you to build modular, reusable and scalable code.

- [Python OOP](#)
- [Classes and Objects](#)
- ['Self' as Default Argument](#)
- [Polymorphism](#)
- [Inheritance](#)
- [Abstraction](#)
- [Encapsulation](#)
- [Iterators](#)

- **Quiz:** [OOP](#)

Exception Handling

In this section, we'll explore Exception Handling explains how Python deals with unexpected errors, enabling us to write fault-tolerant code. We'll cover file handling, including reading from and writing to files.

- [Exception Handling](#)
- [Built-in Exception](#)
- [User defined Exception](#)
- **Quiz:** [Exception Handling](#)

File Handling

In this section, we will cover file handling, including reading from and writing to files.

- [File Handling](#)
- [Read Files](#)
- [Write/Create Files](#)
- [OS Module](#)
- [pathlib Module](#)
- [Directory Management](#)
- **Quiz:** [File Handling](#)

Database Handling

In this section we will learn how to access and work with MySQL and MongoDB databases

- [MongoDB Introduction](#)
- [MySQL Introduction](#)

Packages or Libraries

Python has a huge collection of packages and standard libraries that make development easier. These libraries help with a wide range of tasks and can save a lot of time by providing ready-to-use tools.

Some commonly used types of libraries include:

- [Packages](#)
- [Built-in Modules](#)
- [DSA Libraries](#)
- [GUI Libraries](#)

Data Science

1. Foundational Libraries: These are the libraries that form the base for all data science work. Start here to build a strong foundation.

- [NumPy](#)
- [Pandas](#)
- [Matplotlib](#)

2. Advanced Visualization and Statistical Tools: Once you're comfortable with basic data handling and visualization, move to creating cleaner visuals and performing statistical analysis.

- [Seaborn](#)
- [Statsmodel](#)

3. Machine Learning Libraries: After data manipulation and visualization, learn machine learning, starting with simpler models and moving to advanced ones.

- [Scikit-learn](#)

- [XGBoost /LightGBM](#)

4. Deep Learning Frameworks: If you're interested in AI and deep learning, these libraries will allow you to build and train neural networks.

- [TensorFlow](#) and [Keras](#)
- [PyTorch](#)
- Complete Tutorial on [Data Science](#)

Web Development

1. Core Web Frameworks (Backend Development with Python): These are the tools for building Python-based web applications.

- [Flask](#)
- [Django](#)

2. Database Integration: Learn how to connect Python web frameworks to databases for storing and retrieving data.

- [SQLite](#)
- [SQLAlchemy](#)
- [MySQL](#)

3. Front-End and Backend Integration: Learn how to connect Python backends with front-end technologies to create dynamic, full-stack web applications.

- [Jinja2 \(Flask\)](#)
- [Django Templates](#)

4. API Development: Learn to build APIs (Application Programming Interfaces) for connecting your backend with front-end apps or other services.

- [Flask-RESTful](#)
- [Django REST Framework \(DRF\)](#)
- Complete Tutorial on [Web Development](#)